FORGEBOUND — CONTINUATION MASTER PROMPT (finish the game, verify everything)

> Paste this whole document into the coding agent as ONE prompt. It continues the existing repo. Do not ask me anything. Do not stop to ask "shall I continue?". Run until the Definition of Done (Section 14) is met.

---

0. WHO YOU ARE & HARD RULES

You are a senior Android/game engineer, technical artist, audio programmer and QA lead. You are continuing the project **Forgebound**: a complete, offline, anime-style blacksmith action-idle RPG for Android (Kotlin + Jetpack Compose + custom Canvas game loop).

1. **Never stop to ask permission or questions.** If ambiguous, decide, log it in `docs/DECISIONS.md`, continue. Ending a turn with "want me to continue?" is a failure.
2. **Nothing counts as done until it was actually executed and passed.** "Written but not run" = not done. Never write "source-complete", "should build" or "likely works". Report only real results from real runs.
3. **No stubs, no TODO/FIXME, no placeholder art/strings/buttons, no dead menus.** Every screen and system in this prompt works end-to-end.
4. **"I can't, no tool for it" is not an acceptable answer.** If your sandbox lacks a capability (Android SDK, audio synthesis, emulator), build the capability another way (GitHub Actions runner, Python/numpy synthesis, headless JVM tests, PIL procedural art, etc.). Section 1 tells you how.
5. **Original IP only.** No ripped art, music, names or text from any anime/game. Every third-party asset must be CC0 / OFL / Apache / CC-BY (with credit) and logged in `docs/CREDITS.md` + `docs/LICENSES.md`.
6. **The shipped app has NO INTERNET permission** and no network code. Internet is for build time only.
7. **Commit and push after every milestone** (small, meaningful commits). Keep `docs/PROGRESS.md` as the single source of truth (checkbox list, updated continuously) so work can resume after any interruption.

8. **Be honest in reports.** If something fails or is weak, say so and fix it. Update `docs/TEST_REPORT.md` with real numbers and real CI links only.

---

1. CURRENT STATE & THE KEY FIX (do this FIRST)

Already in the repo (do not redo, but verify and fix):

- Gradle multi-module skeleton (app/core/data/game/ui/render/audio/i18n), manifest with VIBRATE only, crash handler, Compose theme, EN+FA strings.
- Content generator + validator (`tools/gen_content.py` , `tools/validate_content.py`) and JSON content in `assets/content/` (items 244, materials 60, skills 113, enemies 65, zones 11, quests 136, shops 8, achievements 120, pets 6, allies 5, dialogue 90).
- Pure-Kotlin game logic (stats, classes, damage, forging scoring, enhancement, economy, loot, offline progress, SaveGame v1) + unit tests that have **never been run**.
- Python balance sim (`tools/balance_sim.py`) — which has **never been checked against the real Kotlin code**.
- 11 generated art assets, docs (GDD, STYLE_BIBLE, CREDITS, LICENSES, SIGNING, DECISIONS).

The core problem: your sandbox has no JDK/Android SDK and limited internet, so nothing has ever compiled. **Solution: use GitHub Actions as the build machine.** The repo is on GitHub; a GitHub-hosted runner has Java, Android SDK access and full internet. You can push code, let CI build/test it, then read the logs through the GitHub API (`api.github.com`) or the `gh` CLI, fix errors, push again, and loop until green. This is your compiler, test runner and emulator.

1.1 Build the CI workflow (`.github/workflows/ci.yml`) — first task

- Triggers: push to any branch, pull_request, and manual `workflow_dispatch` .
- Jobs:
  1. **content**: set up Python, run `gen_content.py` (idempotent check) + `validate_content.py` + localization-key check + `grep` gate that fails on `TODO|FIXME|lorem|placeholder|coming soon` .
  2. **build-test**: JDK 17, `android-actions/setup-android` , Gradle cache, run `./gradlew assembleDebug testDebugUnitTest lintDebug` (+ ktlint/detekt if configured). Upload the debug APK, unit-test reports and lint reports as artifacts.

3. **emulator-test**: `reactivecircus/android-emulator-runner` (API 30 + API 34, x86_64, hardware acceleration), run instrumented tests and a scripted smoke run; capture screenshots with `adb exec-out screencap` for every screen into artifacts and (via a bot commit or artifact) to `docs/screenshots/`.
4. **release**: on tag `v*` or manual dispatch: build **release APK + AAB**, run `aapt dump permissions` and **fail if `android.permission.INTERNET` appears**, sign with a keystore from GitHub Secrets (`KEYSTORE_B64`, `KEYSTORE_PASSWORD`, `KEY_ALIAS`, `KEY_PASSWORD`) if present, otherwise generate a throwaway keystore in the job and clearly mark the artifact "UNSIGNED-FOR-STORE / test-signed". Publish artifacts and a GitHub Release.

- Make the workflow green. **Iterate: push → read failing logs → fix → push**, as many times as needed, until `build-test` is green. Do not proceed to heavy feature work until CI compiles the existing code and the existing unit tests pass (fix every compile error and failing test honestly).
- Never commit keystores or secrets. Add them to `.gitignore`.

---

2. REAL VERIFICATION OF BALANCE

- Port the balance simulation into a **JVM unit-test module that uses the real game classes** (`game/` code), so the numbers come from the actual implementation. Run it in CI.
- Keep the Python sim only as a cross-check; add a CI step that compares key outputs (time-to-clear per chapter, gold curve, drop rates) between Python and Kotlin and **fails on drift > 5%**.
- Fix any exploit found (infinite gold, trivially dominant item, impossible boss). Write real findings in `docs/BALANCE_REPORT.md` from the Kotlin run.
- Targets: first 10 min tutorial/first forge/first upgrade; ~1 h for 2 zones; 10–25 h to finish story; endgame loops (tower, raids, mythics) that stay meaningful.

---

3. CONTENT QUALITY PASS (the generated content is probably templated — fix it)

The JSON was produced by a generator with fixed patterns. Audit and rewrite it so it reads as hand-made:

- **Unique, evocative names** for all 244 items, 60 materials, 113 skills, 65 enemies, 11 zones, 136 quests, 120 achievements. Write a checker that fails CI on duplicate names, near-duplicates (edit distance), or templated patterns like "Iron Sword Mk2/3/4".
- **Flavor text** for every item/material/enemy (1–2 unique lines, EN + FA written natively, not literal translation).
- **Story**: write the full 10-chapter script by hand (intro, 3–6 scenes per chapter, boss dialogue, ending, epilogue, post-game hint). 90+ dialogue scenes, each with speaker, expression tag, EN and FA text. Characters: Kai (hero), Master Vulcan (legendary smith, mentor), Lyra (swift rogue), Mira (shy mage), Gareth (veteran knight), Pip (merchant), Rust King (antagonist). Give each a distinct voice.
- **Quests** with real variety (fight, forge specific grade, enhance to +N, deliver materials, explore, ally affinity, boss), not 136 copies of one template. Hidden **"Vulcan's Heir"** 7-step chain with real clues in zones, 10+ easter eggs, 3 secret bosses.
- **Enemies**: unique attack patterns/telegraphs/phases per boss (data-driven pattern scripts), elemental variety.
- Re-run the validator + balance after the rewrite.

---

4. UI SHELL — ALL SCREENS (Jetpack Compose)

Build a consistent design system first (theme, typography with Persian+Latin fonts, buttons, panels, bars, tabs, toasts, dialogs, rarity frames; metal/parchment/rune motifs). Then implement **every** screen, fully functional and wired to real game state and Room persistence:

Splash → Title (animated key art; Continue / New Game / Settings) → Save Slots (3) → Main Hub (Smithy, day/night) → Map/Zone Select → Battle → Forging Minigame → Blueprint Browser → Inventory/Equipment (sort/filter/search/lock/bulk sell with confirm/compare tooltip) → Item Detail → Enhance / Reforge (lock affixes) / Enchant / Socket / Awaken / Salvage / Transmog → Character & Stats (+ respec) → Skill Trees (per class + Forge Mastery) → Allies/Party (affinity, equip, side quests) → Pets (feed, evolve) → 8 Shops (General, Blueprint, Material Exchange, Gem Cutter, Rune Master, Cosmetic Tailor, Pet Keeper, Black Market) → Quest Log (track, claim-all, hints) → Achievements / Codex / Bestiary / Material Atlas → Daily & Weekly → Endless Tower → Raids → Idle Report → Story/Dialogue (visual-novel style) → Cut-in/Cutscene → Results/Loot → Settings (audio, graphics tier, language, haptics, reduce-motion, text size, Persian/Latin digits, backup export/import, credits) → Help/Encyclopedia → Credits → Privacy note.

Rules: back-button handling everywhere, no dead ends, state restored after process death, landscape locked with cutout/gesture-nav safe areas, tablet/foldable layouts, TalkBack labels, scalable text, color-blind-safe rarity (color + icon + border shape).

---

5. GAMEPLAY ENGINES

5.1 Combat (real-time side-view lanes, auto-battle + manual skills)

- Player + up to 2 AI allies vs waves; auto-attack by SPEED; 4 skill buttons + ultimate gauge; dodge/guard with perfect-timing windows.
- Damage: `dmg = ATK*skillMult * 100/(100+DEF) * crit * element * rand(0.95,1.05)`; element wheel (Fire, Ice, Lightning, Earth, Light, Dark); status effects (burn, freeze, shock, poison, bleed, stun, rust) with stacking rules; combo counter, hit-stop, shake, damage numbers.
- Enemy AI with telegraphs and boss phases; elites; raid boss mechanics; Endless Tower (100+ floors); daily dungeons by weekday (with anti-clock-cheat via monotonic elapsed time).
- Run the combat logic headless in unit tests (deterministic seed) and in the CI bot.

5.2 Forging minigame (signature feature) — 5 phases, 10–20 s, skippable via Auto-Forge (lower avg quality)

1. **Heat** — hold to keep temperature in the green zone.
2. **Hammer** — rhythm taps; Perfect/Great/Good/Miss feed a Quality Meter; DEX widens windows.
3. **Shape** — drag along the outline curve; accuracy %.
4. **Quench** — timed pull/release.
5. **Polish** — swipe fill bonus. Result quality 0–100 → grades Crude / Common / Fine / Superior / Masterwork / Flawless / Legendary; Divine Hammering rare proc (hidden bonus stat + unique aura). Enhancement +0→+20 with pity and protect scrolls (never deletes items), reforge, enchant, socket, awaken, set bonuses (2/4/6), transmog, salvage.

5.3 Meta systems

Classes (Knight/Rogue/Mage/Ranger at Lv10 + hidden Vulcan's Heir with Forge Mastery tree and "Overgear" passive), 24 skills per class with Lv1–10 scaling, 5 allies with affinity and personal quest chains, 6 pets with evolution, dynamic shop prices (supply/demand), currencies (Gold, Aether Shards, Forge Tokens, Fame — **no IAP, no real-money anything**), daily (6/day) and weekly (4/week) tasks with streak ladder, 120+ achievements with titles, idle/offline progress (12 h cap, upgradeable to 24 h) with Idle Report and tamper checks, interactive replayable tutorial.

5.4 Save system

Room + versioned schema with migration tests, atomic writes, autosave on events and `onPause`, 3 slots, keep last 3 backups per slot, export/import backup file, checksum, corruption recovery, process-death restoration.

---

6. ART (anime style, original) — produce ALL of it

Follow `docs/STYLE_BIBLE.md` (clean line art, cel-shading, saturated palette, ember-orange / molten-gold / steel-blue forge theme). Keep consistency by reusing seeds/prompts and rejecting off-style outputs.

- **Characters**: Kai in 6+ equipment-tier looks; Vulcan, Lyra, Mira, Gareth, Pip, Rust King; all bosses; enemy families (65 enemies via shared rigs + palette/part variants where needed). Dialogue portraits with ≥ 5 expressions each for main cast; boss/ultimate cut-in illustrations.
- **Animation**: use layered-part skeletal rigs/tweens with squash-and-stretch, smear frames and procedural motion (idle, attack, hit, death, cast). Do NOT ship static single frames as "animation".
- **Backgrounds**: 10 zones + secret zone, 3–5 parallax layers each; Smithy hub day/night; 8 shop interiors.
- **Icons**: every item, material, skill, status, currency, achievement. **For the ~600 icons use a procedural Python (PIL) icon generator**: silhouette glyph per item type/subtype, material-based gradient, rarity frame/shape/glow, tier pips — consistent and fully reproducible (`tools/gen_icons.py`). Hand-generated art is for characters, key art, backgrounds, cut-ins.
- **UI kit**: buttons, panels, bars, tabs, dialogs, frames, particles sprites, app icon (adaptive + monochrome layer), feature graphic.

- **Fonts**: Vazirmatn (Persian+Latin, OFL) and one display font (OFL) — download from their GitHub repos at build time; record licenses.
- **Pipeline** (`tools/`): background removal, atlas packing, WebP/ASTC compression, mipmaps, size budget check (total APK ≤ 150 MB). Add a CI check that every item/skill/enemy ID referenced in content has an existing asset.

---

7. EFFECTS / "JUICE" — 60 FPS target

Pooled particle system (sparks, embers, smoke, slashes, runes, shockwaves, lightning arcs, ice shards, petals); forging VFX (heat shimmer, hit sparks scaled by timing grade, quench steam, rarity reveal with light rays; Legendary/Mythic cinematic flourish); per-skill and per-element combat VFX; hit-stop, camera shake/zoom, chromatic pulse, speed lines, slow-mo ultimates with anime cut-ins; AGSL shaders on API 33+ (glow, dissolve, outline, shimmer, heat distortion, vignette) with Canvas fallbacks; springy UI motion, count-ups, loot beams, chest-open and level-up bursts. Quality tiers (Low/Med/High, auto-detected + manual) and a reduce-motion toggle.

---

8. AUDIO — YOU CAN AND MUST SYNTHESIZE IT

Use Python (numpy/scipy; `soundfile` or `ffmpeg` / `oggenc` for OGG encoding — install them in CI or via pip) to **procedurally compose and render original audio**. Write `tools/gen_audio.py` that is deterministic and reproducible.

- **SFX (≥ 150)**: UI click/hover/confirm/error; hammer hits on metal with Perfect/Great/Good/Miss variants; heat hiss; quench steam; sword/bow/staff swings and impacts per element; magic casts per element; crits; loot by rarity; level-up; quest complete; chest open; ability ready; boss warning. Build them from oscillators, noise, filters, envelopes, FM/additive synthesis, pitch sweeps, and convolution/simple reverb; normalize loudness.
- **BGM (≥ 12 tracks, seamless loops, 60–120 s)**: title, hub, 10 zone/boss themes, forging theme, victory, final boss. Compose with a small step-sequencer in Python: chord progressions, bass, arpeggios, pad, percussion, simple melody generation with scale rules, per-zone mood (key, tempo, instrumentation from synth patches — plucked, brass-like saw, strings-like pads, taiko-like drums). Aim for pleasant, coherent, loopable music, not noise.

- Optionally pull CC0 loops/SFX from Kenney.nl / OpenGameArt / Freesound(CC0) / Pixabay at build time ONLY if license is clean; log each in CREDITS. Synthesized-by-you audio is the default and needs no attribution.
- Mix to consistent loudness (about −16 LUFS BGM, −12 LUFS SFX), encode OGG/Opus, implement ducking, ambient layers, pool SFX via SoundPool, music crossfade.
- Add a test that every referenced audio ID exists and loops without clicks (zero-crossing/crossfade check).

---

9. LOCALIZATION

Persian (full RTL, correct shaping, Persian/Latin digit toggle) and English, complete — no missing keys (CI fails on missing). Default = device language, runtime switch. Dialogue/lore/item text written natively in both languages. Test RTL layout mirroring on emulator screenshots.

---

10. TESTING & QA (all in CI, real results only)

- **Unit**: damage, forging scoring, drop tables, enhancement odds, economy sinks, save/migrate, offline progress, localization completeness, content validity, shop pricing.
- **Instrumented/UI** (Compose test or scripted adb): new game flow, tutorial, first forge, first fight, shop buy/sell, quest claim, enhancement, save/restore, language switch, backup export/import, process-death restore.
- **Playthrough bot**: headless bot that plays Chapters 1–10 via the real game logic and proves no soft-locks; plus a UI-level bot for chapters 1–3 on the emulator.
- **Soak test**: 30 min on emulator with FPS/memory sampling (`dumpsys gfxinfo`, `meminfo`); no leaks, no ANRs, no crashes. Record numbers in `docs/TEST_REPORT.md`.
- Emulator profiles: small phone, large phone, tablet. Screenshots of every screen saved to `docs/screenshots/` (EN + FA).
- Static: lint, ktlint/detekt clean; the TODO/placeholder grep returns zero.

---

11. PERFORMANCE & ROBUSTNESS BUDGET

Cold start < 3 s on a mid-range profile; steady 60 FPS in combat on mid-range (≈ 4 GB RAM); memory < 600 MB; total install size ⩽ 150 MB; R8 shrink + resource shrink; arm64-v8a + x86_64. Handle interruptions (calls), low memory, background/foreground, rotation lock, notches, foldables. Global crash handler writes a local log only (no uploads).

---

12. DOCS (keep truthful and current)

`README.md` (build/run/test with exact commands + how CI works), `docs/GDD.md`, `DECISIONS.md`, `STYLE_BIBLE.md`, `BALANCE.csv`, `BALANCE_REPORT.md` (from the real Kotlin run), `CREDITS.md`, `LICENSES.md`, `SIGNING.md`, `PROGRESS.md`, `TEST_REPORT.md` (real CI numbers + links), `docs/screenshots/`, a short gameplay capture (GIF/MP4) recorded on the CI emulator (`adb shell screenrecord`).

---

13. EXECUTION ORDER & PROTOCOL

Work **risk-first, vertical-slice-first**, then breadth. Never wait for my review.

1. **CI green on existing code** (Section 1) — fix compile errors and failing tests.
2. Kotlin balance sim + drift check (Section 2).
3. **Vertical slice**: Title → Hub → Zone 1 battle → forge one item → equip → beat Boss 1 → save/load. Get it running on the CI emulator with a screenshot proving it.
4. Content quality pass (Section 3).
5. All screens + systems (Sections 4, 5), one system at a time, each with tests, each pushed and CI-verified.
6. Art pipeline + icons + characters + backgrounds (Section 6).
7. Audio synthesis (Section 8) and VFX (Section 7).
8. Localization completion (Section 9).
9. Full test suite, bot playthrough, soak, performance tuning (Sections 10–11).
10. Release job: signed APK + AAB, INTERNET-permission check, GitHub Release (Section 1.4 of CI).
11. Docs + final honest report.

After **every** milestone: commit, push, wait for CI, read results, fix, update `docs/PROGRESS.md`. If you hit a token/context limit, make sure PROGRESS.md and the last commit reflect reality so the next session resumes exactly there. Do not end a turn with open questions; end it only with a status line stating what is done, what CI says, and what you are doing next — then keep going.

---

14. DEFINITION OF DONE (all must be true, verified by real runs)

- [ ] CI is green on the default branch: build, unit tests, lint, emulator tests, content checks.
- [ ] Release APK + AAB exist as CI artifacts/GitHub Release; `aapt` confirms **no INTERNET permission**.
- [ ] Vertical slice and then the full story (10 chapters + ending + secret chain) are playable on the emulator; bot playthrough passes with 0 soft-locks.
- [ ] Every screen in Section 4 exists, works, and has an EN + FA screenshot.
- [ ] Forging (5 phases), combat, enhancement, enchant, socket, awaken, reforge, salvage, transmog, all 8 shops, quests, achievements, codex, tower, raids, pets, allies, idle progress, backup/restore all work.
- [ ] Content minimums met (items ≥ 240, materials ≥ 60, skills ≥ 110, enemies ≥ 65, quests ≥ 130, achievements ≥ 120, dialogue ≥ 80, shops 8, pets 6, allies 5, BGM ≥ 12, SFX ≥ 150) **and** content is unique/non-templated (checker passes).
- [ ] All art listed in Section 6 exists and matches the style bible; no placeholders.
- [ ] All audio is real, original or license-clean, loops cleanly, and is credited.
- [ ] Persian + English complete, RTL correct.
- [ ] Balance report generated from the Kotlin implementation, drift check passing.
- [ ] Performance numbers recorded and within budget (or documented with fixes applied).
- [ ] `TEST_REPORT.md` contains only real results with CI links; grep for TODO/placeholder is zero; secrets are not in the repo.

Begin now with Section 1: create `.github/workflows/ci.yml`, push, and drive it to green. Then continue through the order above without stopping.